A Computer-Vision-Based Bikeability Score

Bünyamin Aydemir, Gonzalo Guerrero Salazar, and Hendrick Fischer

***Abstract*—**

***Index Terms*—Bikeability, computer vision, zero-shot classification, vision-language models, CLIP, surface classification, object detection, GPX.**

I. Introduction

II. Research Approach and Methodology

III. Objectives and Constraints

Manuscript created July 2026. The authors are students at the Baden-Württemberg Cooperative State University (DHBW) within the Computer Vision course.

IV. Ground Detection: Zero-Shot Classification of the Road Surface

Ground detection determines the ridden surface class for every video frame and thereby provides the first of the three sub-components of the bikeability score. Since each frame corresponds to exactly *one* surface class, the task is formulated as *whole-image classification* (following the principle "whole image = one class") rather than semantic segmentation. The input images originate from a camera mounted at the front of the bicycle handlebar and are sampled at one frame per second.

A. Task Type and Model Class

Because neither annotated target classes nor a project-specific training set are available in field operation, the surface is determined *training-free* (zero-shot). This leads to the choice of *vision-language models* (VLM) of the CLIP family [1], which select the matching class from an image and a set of text prompts without any fine-tuning. Classical CNNs trained on ImageNet do not know application-specific classes such as "gravel path" and are therefore ruled out.

A distinction that is frequently confused in the current literature is central here: *linear probing* of self-supervised features (e.g., DINOv2/DINOv3 [2]) is *not* a zero-shot method. A linear classifier on frozen features requires trained labels of the target classes and is therefore *supervised*. Such a method is deliberately not counted as zero-shot here; at most it could serve as an explicitly declared *non-zero-shot reference point*.

B. Surface Classes and Prompt Ensembling

Four cycling-relevant classes are distinguished: **Cycleway** (paved bike path), **Road** (paved/asphalt road), **Gravel** (gravel/crushed stone), and **Unpaved** (dirt/field track). Since *Cycleway* and *Road* both consist of smooth asphalt on the surface, the text prompts deliberately emphasize the *distinguishing context* (narrow, structurally separated, bicycle pictogram vs. wide, cars, center line, multiple lanes) rather than only the texture. For each class, several English descriptions are used, and their text embeddings are averaged and normalized (*prompt ensembling* as in the CLIP paper [1]), which makes the class embeddings more discriminative. An image is classified via the cosine similarity to these class embeddings; a softmax yields class probabilities.

C. Compared Models

To ensure a fair comparison, all models are accessed through the unified `open_clip` API [3]. A range from *very light* to *medium-sized* is deliberately considered – all local/edge capable, no closed "killer model". Table I summarizes the considered architectures.

D. Metrics

The benchmark evaluates the models along two axes: *accuracy* and *latency*.

Table I
Models compared in the zero-shot benchmark.

Model	Characteristic
CLIP ViT-B/32, B/16, L/14 [1]	Classical reference
OpenCLIP (LAION-2B, DataComp-XL) [3], [4]	Dataset influence
SigLIP ViT-B/16 [5]	Efficient zero-shot standard
MobileCLIP S0, S2 [6]	Latency-optimized
EVA02-B/16 [7]	Open zero-shot default

Figure 1. Accuracy (macro-F1) vs. latency (ms/frame, batch size 1). Point size $\propto$ number of parameters; the dashed line marks the Pareto front. *Figure to follow.*

Figure 2. Row-normalized confusion matrices per model. They show which surfaces are confused (typically Gravel $\leftrightarrow$ Unpaved as well as Road $\leftrightarrow$ Cycleway). *Figure to follow.*

a) Accuracy: Since the classes are imbalanced, in addition to accuracy we mainly report the *balanced accuracy* (mean of the per-class recalls) and the *macro-F1*. In addition, a *confusion matrix*, a *bootstrap confidence interval* (95 %, 2000 resamples) on the accuracy, and the *McNemar test* for pairwise model comparison are computed per model. The McNemar test checks on the discordant predictions whether an accuracy difference is *statistically significant* ($p < 0.05$) and not merely noise on a small test set.

b) Latency: On fixed hardware, the pure inference latency is measured at *batch size 1* (real use case: video at the handlebar): several warm-up runs, followed by N measured single inferences including GPU synchronization. The mean, standard deviation, the 95th percentile, and the throughput in frames per second (FPS) are reported.

E. Result Presentation

The central overall representation is a scatter plot of *accuracy vs. latency* (Fig. 1). Each point is a model (point size $\propto$ number of parameters); the drawn *Pareto front* connects the models that achieve the best accuracy for a given latency – i.e., the best trade-off for use at the bicycle handlebar. In practice, **MobileCLIP** variants often lie on the Pareto front, as they offer a very favorable latency-accuracy ratio without requiring an oversized model.

For reproducibility: latency values are strongly hardware-dependent, therefore the concrete computing device (CPU/GPU model, RAM) as well as the used `DEVICE` are documented. All models use the same full image, the same prompt ensembling, and the same `open_clip` pipeline; observed differences therefore stem from architecture and training data, not from the evaluation.

V. Object Detection

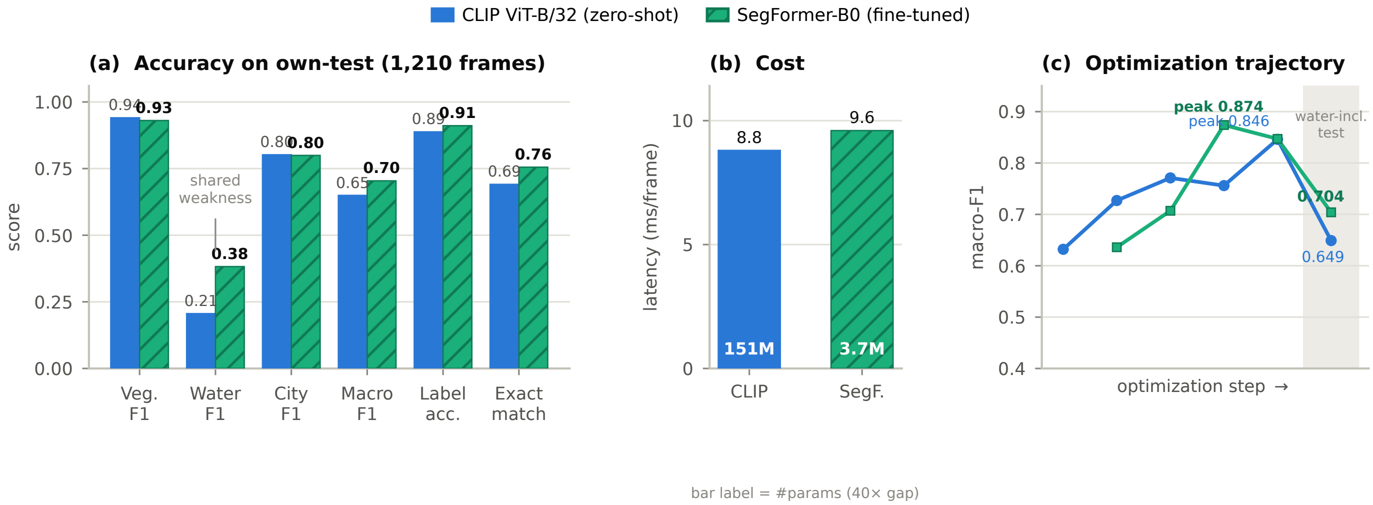

Figure 3. Environment-model comparison on the own-frames test set (1,210 ride-disjoint cyclist-POV frames; 983 vegetation / 40 water / 307 city positives), decision thresholds tuned on the validation split only. **(a)** Per-class and aggregate accuracy: the fine-tuned SegFormer wins macro-F1, label-accuracy, exact-match and water, while CLIP edges vegetation; water is the shared weakness. **(b)** Both models run at comparable latency, but SegFormer uses 40× fewer parameters (Apple MPS, warmed, batch 1; bar label = #params). **(c)** Macro-F1 across the optimization steps of Tables II and III; the drop at the shaded stage is the harder water-inclusive test, not a regression.

VI. Environment Detection: Zero-Shot vs. Fine-Tuned Semantic Segmentation

Environment detection classifies the surroundings of every frame and supplies the environment sub-component, which carries the *largest* weight in the bikeability score ($w_e = 0.45$, Eq. (2)). Unlike ground detection, the task is *multi-label whole-image* classification: a frame may show vegetation, water and a built-up scene at once – or none of them. We compare two paradigms: a training-free *vision–language model* (CLIP [1], following recent street-view work [8]) and a fine-tuned *semantic-segmentation* transformer (SegFormer [9]) whose per-class pixel-area fractions feed the sub-score directly (Eq. (3)).

A. Task Type and Model Class

The two model classes answer the multi-label question differently. *Zero-shot CLIP* derives a per-class present-probability from prompt pairs and needs no application-specific training data, as demonstrated for street-view imagery in [8]. *Fine-tuned SegFormer* instead predicts a pixel mask whose class-area fractions are thresholded into presence; these fractions are richer than a scalar probability and are exactly the signal the beauty score consumes [10]. Both models are light enough for local/edge inference.

B. Environment Classes and Taxonomy

Three cycling-relevant classes are distinguished: **Vegetation** (trees, grass, fields), **Water** (river, lake, canal or sea covering more than about 2% of the frame) and **City** (buildings, industry, or a street with houses). The taxonomy was deliberately simplified during development from five to three classes: an *industry* class was folded into *City*, and the fuzzy *forest/open-field* boundary – the weakest class for both models – was merged into *Vegetation*. This merge was the single most impactful design change (Table III, S9). The fine-tuned model additionally learns an explicit *“other”* background class, so that background pixels are not force-assigned to an environment class. Water is intrinsically rare in street-level imagery, which drives much of the optimization below.

Table II
CLIP zero-shot optimization steps (own-frames test; Δ is the macro-F1 change over the previous adopted step). Final config: ViT-B/32, prompt ensembling, per-class thresholds veg 0.035 / water 0.39 / city 0.73.

#	Optimization step	Effect / decision
C1	Aligned prompt pairs, global threshold	baseline
C2	Ride-disjoint val/test split	leakage-free tuning
C3	Per-class threshold calibration	**+0.095**; top lever
C4	Prompt ensembling, 6–8 templates [1]	+0.044; adopted
C5	Larger backbone ViT-L/14	−0.015, 24× slower; rejected
C6	3-class taxonomy (veg. merge)	best config
C7	Water-inclusive final test	water F1 .62→.21: artifact

C. Zero-Shot CLIP: Optimization

Table II lists the zero-shot track. Two levers dominated: *per-class threshold calibration* on a leakage-free validation split (+0.095 macro-F1) and *prompt ensembling* [1], which averages the text embeddings of 6–8 templates per class (+0.044). Model capacity did *not* help – ViT-L/14 was worse and ∼24× slower to encode, and was rejected. The final water-inclusive test then revealed that CLIP’s apparent water strength was a small-validation artifact (water F1 0.615 on validation vs. 0.206 on the held-out test rides).

Table III
SEGFORMER FINE-TUNING OPTIMIZATION STEPS. "DEV" = MAPILLARY MASK-DERIVED VALIDATION; "OWN-TEST" = CYCLIST-POV FRAMES. FINAL CONFIG: B0 ADE20K BASE, 3 CLASSES + "OTHER", THRESHOLDS VEG 0.25 / WATER 10^{-4} / CITY 0.0056.

#	Optimization step	Effect / decision
S1	Cityscapes-B0, random Mapillary [11]	water F1 0.00 (baseline)
S2	Learning rate $6\times10^{-5}\to2\times10^{-4}$	+0.05; water still 0
S3–4	Balanced + water ×2–6 + ADE water [12]	water IoU 0→0.70: key fix
S5	Base swap → ADE20K (water prior) [12]	water IoU 0.778, mIoU 0.857
S6	Copy-paste water augmentation	water IoU −0.11; rejected
S8	Explicit "other" background class	label-acc .65→.86
S9	3-class taxonomy retrain	own-test macro 0.874; deployed
S10	Fine-tuning ablation (no-FT)	FT worth +0.027 (city +0.05)
S11	Fine threshold grid ($\to 10^{-4}$) + final test	**macro 0.704: wins**

D. Fine-Tuned SegFormer: Optimization

Table III lists the fine-tuning track. The decisive gains were about *data*, not hyperparameters: water is so rare that a random Mapillary sample [11] contained almost none, so class-balanced sampling with water oversampling plus 258 added ADE20K water scenes [12] lifted water IoU from 0 to 0.70. Swapping the encoder to an ADE20K-pretrained base [12], [13], which already carries a water prior, and adding the explicit *"other"* class raised own-test label-accuracy from 0.649 to 0.863. A fine-tuning ablation (S10) isolates the value of transfer learning itself: +0.027 macro-F1 over the raw checkpoint, concentrated in *City*.

E. Metrics and Protocol

The two models are evaluated as multi-label classifiers on **macro-F1** (unweighted mean of the per-class F1), **label-accuracy** (mean per-(frame, class) correctness), **exact-match** (fraction of fully-correct label vectors) and **per-class F1**; segmentation quality is tracked via mIoU during training. The headline evaluation set is **1,210** cyclist-POV frames that are ride-disjoint from all tuning (983 vegetation / 40 water / 307 city positives), and every decision threshold is fitted on a separate validation split only, so the test rides are never seen during calibration.

F. Results and Decision

Figure 3 summarizes the head-to-head. The fine-tuned SegFormer wins *every* aggregate metric (macro-F1 0.704 vs. 0.649; label-accuracy 0.911 vs. 0.888; exact-match 0.755 vs. 0.691) while using $40\times$ fewer parameters (3.7 M vs. 151 M) at comparable latency (Fig. 3b). A per-class *hybrid* routing of the two models was evaluated and rejected: the best routing gained ≤ 0.005 macro-F1 – not worth carrying a second, far larger model. *Water remains the shared limitation*: at best F1 0.382, the model is *blind* rather than miscalibrated (68% of true-water frames receive almost no water pixels), because in-domain water training data is scarce. The fine-tuned SegFormer is therefore the deployed environment model, and its per-class area fractions feed Eq. (3) directly, which CLIP's probabilities cannot.

VII. Computation of the Bikeability Score

The bikeability score combines the three sub-components – surface (ground detection), environment (environment detection), and objects (object detection) – into a single value in the interval $[0, 100]$. Each model output is first transformed into a *sub-score* in $[0, 1]$; the three sub-scores are then combined in a weighted manner.

A. Overall Formula

For each evaluated frame, the score A results from the weighted mean of the three sub-scores:

$$A = 100 \cdot \Big(w_g\, S_{\text{ground}} + w_e\, S_{\text{env}} + w_o\, S_{\text{object}}\Big), \tag{1}$$

with the weights

$$w_g = 0.20, \quad w_e = 0.45, \quad w_o = 0.35, \quad w_g + w_e + w_o = 1. \tag{2}$$

Since all sub-scores lie in $[0, 1]$ and the weights sum to 1, the score is *guaranteed* to be bounded to $[0, 100]$.

B. Ground and Environment Sub-Score

The surface and environment sub-scores result from a direct lookup of the respective class quality. If, instead of a single class, a share distribution $\{(k, p_k)\}$ is present, the weighted mean of the class qualities is formed:

$$S_{\text{ground}} = \sum_k q_k^g\, p_k, \qquad S_{\text{env}} = \sum_k q_k^e\, p_k, \tag{3}$$

where p_k is the share of class k ($\sum_k p_k = 1$) and $q_k^g, q_k^e \in [0, 1]$ are the stored quality values. Table IV lists the (heuristic) quality values used.

C. Object Sub-Score

The object sub-score evaluates the detected object counts n_k (e.g., cars, buses, cyclists). Via a *saturation function*, a weighted object density is mapped onto $(0, 1]$:

$$S_{\text{object}} = \exp\Big(-\max\big(\rho, 0\big)\Big), \qquad \rho = \frac{1}{N}\sum_k \lambda_k\, n_k, \tag{4}$$

with the number of evaluated frames N and the penalty factors λ_k from Table IV. Positive penalty factors (e.g., cars, buses) lower the sub-score, while bicycle-friendly objects such as other cyclists receive a *negative* factor and raise the score slightly. The exponential form is robust against outliers: a single overcrowded frame does not dominate the overall score.

Table IV
Class quality values for the ground and environment sub-score as well as object penalty factors.

Component	**Class**	**Value**
Ground q_k^g	Cycleway	1.0
	Road	0.7
	Gravel	0.3
	Unpaved	0.1
Environment q_k^e	Vegetation	1.0
	Water	1.0
	City	0.4
Object penalty λ_k	Car	$+1.0$
	Bus	$+2.0$
	Bicycle	-0.2

Figure 4. Bikeability score colored along the GPX route. The frame scores are merged with the GPX track points via the absolute timestamp. *Figure to follow.*

D. Route Evaluation and Aggregation

To evaluate a complete ride, the video is sampled at fixed intervals (every 5 s, i.e., every $\lfloor \text{FPS} \cdot 5 \rfloor$-th frame) and a score is computed for each frame according to Eq. (1). Each row is stored in a CSV with a timestamp (seconds from video start, `HH:MM:SS`, and optionally an absolute ISO timestamp). Via the absolute timestamp, the frame scores can then be *merged* with the GPX track points of the ride and located geographically. The *average* bikeability score of the route results as the mean over all M evaluated frames:

$$\bar{A} = \frac{1}{M}\sum_{i=1}^{M} A_i. \tag{5}$$

Figure 4 shows the final result: the bikeability score colored along the GPX route, which visualizes the section-wise cycling-friendliness of the ridden route.

VIII. Conclusion

References

[1] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, “Learning transferable visual models from natural language supervision,” in *Proc. Int. Conf. Machine Learning (ICML)*, 2021, pp. 8748–8763.

[2] M. Oquab, T. Darcet, T. Moutakanni *et al.*, “DINOv2: Learning robust visual features without supervision,” *Trans. Machine Learning Research (TMLR)*, 2024.

[3] G. Ilharco, M. Wortsman, R. Wightman, C. Gordon, N. Carlini, R. Taori, A. Dave, V. Shankar, H. Namkoong, J. Miller, H. Hajishirzi, A. Farhadi, and L. Schmidt, “OpenCLIP,” Zenodo, 2021, https://github.com/mlfoundations/open_clip.

[4] S. Y. Gadre, G. Ilharco, A. Fang *et al.*, “DataComp: In search of the next generation of multimodal datasets,” in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[5] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer, “Sigmoid loss for language image pre-training,” in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2023, pp. 11 975–11 986.

[6] P. K. A. Vasu, H. Pouransari, F. Faghri, R. Vemulapalli, and O. Tuzel, “MobileCLIP: Fast image-text models through multi-modal reinforced training,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2024.

[7] Q. Sun, Y. Fang, L. Wu, X. Wang, and Y. Cao, “EVA-CLIP: Improved training techniques for CLIP at scale,” *arXiv preprint arXiv:2303.15389*, 2023.

[8] W. Huang, J. Wang, and G. Cong, “Zero-shot urban function inference with street view images through prompting a pretrained vision-language model,” *Int. J. Geographical Information Science (IJGIS)*, vol. 38, no. 7, pp. 1414–1442, 2024.

[9] E. Xie, W. Wang, Z. Yu, A. Anandkumar, J. M. Alvarez, and P. Luo, “SegFormer: Simple and efficient design for semantic segmentation with transformers,” in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2021, pp. 12 077–12 090.

[10] D. H. Lee, H. Y. Park, and J. Lee, “A review on recent deep learning-based semantic segmentation for urban greenness measurement,” *Sensors*, vol. 24, no. 7, p. 2245, 2024.

[11] G. Neuhold, T. Ollmann, S. Rota Bulò, and P. Kontschieder, "The Mapillary Vistas dataset for semantic understanding of street scenes," in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, 2017, pp. 4990–4999.

[12] B. Zhou, H. Zhao, X. Puig, T. Xiao, S. Fidler, A. Barriuso, and A. Torralba, "Semantic understanding of scenes through the ADE20K dataset," *Int. J. Computer Vision (IJCV)*, vol. 127, no. 3, pp. 302–321, 2019.

[13] T. Cornille and N. Rogge, "Fine-tune a semantic segmentation model with a custom dataset," Hugging Face Blog, 2022, https://huggingface.co/blog/fine-tune-segformer.